

난독화된 한글 리뷰 복원 AI 프로젝트

A팀 보고서

1. 데이터 이해

다음 내용을 포함한다.

- 난독화 리뷰의 특징
 - 난독화된 텍스트는 원본 텍스트의 의미를 유추할 수 있을 정도로 음운적 유사성을 띠지만, 형태소 분석기나 일반적인 단어 사전을 무력화하는 방식으로 노이즈가 삽입되어 있다.
주로 형태를 훼손하면서도 텍스트의 총 길이는 거의 변하지 않는 1:1 치환 / 변형 방식을 쓴다.
- train/test 차이
 - 학습용 데이터(Train): 총 11,263개로 모델이 정답을 보고 학습할 수 있도록 난독화 텍스트(input)와 원문 텍스트(output) 쌍으로 구성
 - 평가용 데이터(Test): 1,689개이며 모델의 최종 성능을 추론하기 위해 난독화 텍스트(input)만 존재
- 평균 문장 길이
 - 원문 리뷰(output) 평균 길이: 93.12자
 - 난독화 리뷰(input) 평균 길이: 93.09자
 - 텍스트 길이 범위: 최소 1자 ~ 최대 1,381자
- 자주 등장하는 난독화 유형

다음과 같은 4가지 주요 패턴이 복합적으로 나타남.

- 불필요한 받침 추가: 원래 받침이 없는 글자에 무작위로 종성을 추가하여 형태를 변형 (예: 절대 -> 절테)
- 된소리 및 거센소리 변형: 평음을 파열음, 마찰음 계열의 된소리나 거센소리로 바꿈 (예: 장점 -> 짱점)
- 모음 변형: 시각적으로 비슷하거나 유사 발음을 유도하는 다른 모음으로 치환 (예: 스위트룸 -> 쓰윽튼룸)
- 연음 법칙 및 띄어쓰기 훼손: 발음 나는 대로 표기하거나 띄어쓰기를 무시하여 단어의 경계를 무너뜨림 (예: 잠만 자고 갈 때 -> 잠 많 작꼬 갠 태)
- 문자 단위 평가 방식
 - 위의 난독화 패턴들로 인해 단어(Token)와 띄어쓰기 경계가 심각하게 훼손되어 있음.
 - 기존의 단어 단위 일치 여부로 평가를 진행할 경우: 대부분의 문장이 0점으로 처리됨
 - 본 프로젝트에서는 복원된 텍스트가 원문과 글자 단위로 얼마나 일치하는지를 정밀하게 측정할 수 있는 **Character-level F1 Score** 지표 사용

2. 전체 실행 흐름 설명

난독화 텍스트 복원 파이프라인은 다음과 같은 단계로 진행

- 데이터 불러오기
 - 구글 드라이브에 저장된 원시 CSV 파일(train, test)을 Pandas 데이터프레임으로 메모리에 로드하는 단계.
 - 텍스트 데이터를 파이썬에서 프로그래밍 방식으로 조작하고 분석할 수 있는 2차원 표 형태로 구조화하기 위해 반드시 필요함
- 텍스트 전처리 (및 데이터 증강)
 - 결측치 확인, 텍스트 길이 분석을 통해 이상치를 점검하고, 모델의 일반화 성능을 높이기 위해 데이터 증강(Data Augmentation)을 수행
 - 실제 코드에서는 3,000개의 데이터에 특수문자 삽입 및 무작위 띄어쓰기 노이즈를 추가하여 총 14,263개의 학습 데이터를 확보함.
 - 이는 모델이 새로운 패턴의 난독화 데이터에 직면했을 때 과적합(Overfitting)을 방지하고 대응력을 높일 수 있게함
- Dictionary 구축
 - 데이터를 AI 모델에만 의존하지 않고, 직관적이고 고정적인 패턴을 잡아내기 위해 단어 간 1:1 매칭 사전을 만드는 단계
 - Rule-based 방식의 기준점(Baseline) 역할을 하며, 빈도수가 높은 단순 오타자를 연산 리소스 낭비 없이 빠르게 복원하기 위해 필요함
- Tokenizer 구성
 - 자연어 형태의 리뷰 데이터를 LLM(대형 언어 모델)이 연산할 수 있는 숫자 형태의 ID 값으로 변환하는 단계.
 - 한국어의 특성을 반영하여 의미 있는 최소 단위(Sub-word 등)로 텍스트를 분절하여 모델의 입력 형식에 맞추는 필수 전처리 과정
- Dataset 구성
 - Pandas 데이터프레임을 HuggingFace 라이브러리의 Dataset 객체로 변환하고, 학습 세트와 검증 세트(80:20)로 분할
 - 모델이 데이터를 미니 배치 단위로 효율적으로 병렬 학습할 수 있게 만들고, 학습 도중 자체적으로 성능을 평가할 수 있는 분리된 데이터를 마련할 수 있게함.
- 모델 정의
 - 사전 학습(Pre-trained)된 언어 모델(Encoder-Decoder 아키텍처)의 가중치와 구조를 불러오는 단계
 - 모델을 처음부터 학습시키는 것은 리소스가 부족하므로, 이미 한국어 문법과 맥락을 이해하고 있는 모델을 가져와 프로젝트 목적에 맞게 사용할 준비를 함
- 학습 (Fine-tuning)
 - 구성된 학습 데이터(난독화-원문 텍스트 쌍)를 모델에 반복적으로 주입하여, 난독화 패턴을 원래의 단어로 복원하도록 가중치를 미세 조정(Fine-tuning)하는 단계
 - 프로젝트의 핵심 목표인 '복원 능력'을 모델에 실질적으로 부여하는 과정
- 검증 (Validation)
 - 학습 단계 중간마다 검증용 데이터(Validation Set)를 사용하여 모델의 Character F1 Score를 측정
 - 모델이 학습 데이터에만 억지로 끼워 맞춰지는 과적합 상태인지 확인하고, 학습 중단 시점이나 하이퍼파라미터의 적절성을 평가함
- 테스트 추론
 - 학습이 완료된 최적의 모델에 정답이 없는 1,689개의 Test 데이터(input)를 입력하여 복원된 텍스트를 생성(Generation)하는 단계
 - 실제 과제의 목표인 미지의 난독화 리뷰를 해결하는 최종 테스트 과정
- 제출 파일 생성
 - 모델이 추론해 낸 복원 텍스트 결과물을 대회 및 평가 기준이 요구하는 양식(sample_submission.csv)에 맞추어 저장하는 단계

- 분석 결과를 최종 산출물로 만들어 평가 시스템에 제출하기 위한 마무리 담당

3-1. Dictionary Baseline 설명

다음 내용을 포함한다.

문자열 치환 방식

- 입력 문장을 공백 기준으로 분할(Tokenize)하여 어절 단위로 처리함.
- 사전(Dictionary) 내에 분할된 단어(Key)가 존재하는지 확인하여, 존재할 경우 대응되는 원문(Value)으로 치환(Replace)함.
- 사전에 없는 미등록 단어일 경우 치환하지 않고 원본 토큰(Token)을 그대로 반환함.

dictionary 구성 방식

- 학습 데이터의 난독화된 단어(Input)와 원문(Output)을 쌍으로 묶어 매핑함.
- zip() 이용해 난독화된 단어(input)과 원문(output)을 쌍으로 묶음

(input) 별 한 게토 앓갸땡. 왜 싸람듯릭 펄 1개를 준논징 킅킅폰... :
(output) 별 한 개도 아갸다. 왜 사람들이 별 1개를 주는지 꺼어본...

- 문장을 공백 기준으로 토큰화한 뒤, 난독화된 단어를 키(Key)로, 원문 단어를 값(Value)으로 지정하여 딕셔너리를 구축함.
 - input과 output을 공백 기준으로 분할 - 토큰화

(input) 옹끼카 : (ouput) 여기가

- zip()으로 난독화된 단어를 Key, 원문 단어를 Value로 하는 딕셔너리를 생성
- 개선된 구성 방식에서는 정규 표현식(Regular expression)을 활용해 한글 완성형, 영어 대소문자, 숫자, 공백을 제외한 노이즈(Noise)를 제거한 후 사전을 재구성함.

exact matching 개념

- 입력된 단어와 사전의 키 값이 특수기호, 공백, 글자 구성까지 단 1비트의 오차 없이 100% 동일해야만 복원이 수행되는 가장 엄격한 매칭 방식

rule-based 방식의 한계

- 유연성 부족 : 띄어쓰기 변화, 오타, 특수문자 조합 등이 조금만 달라져도 완전히 다른 토큰으로 인식하여 매칭에 실패함.
- 신조어나 사전에 정의되지 않은 새로운 난독화 패턴이 등장할 경우 대응이 불가능함.

- 단어를 독립적으로 처리하므로, 문장 전체의 의미나 주변 단어와의 관계를 전혀 고려하지 못함.
- 사용자가 생성하는 새로운 단어나 난독화 방식의 조합은 무한대에 가까우므로, 이를 모두 사전으로 정의하는 것은 현실적으로 불가능함.

새로운 표현에 취약한 이유

- 단어 간의 의미적 연결성을 유추하지 못하고, 형태가 약간만 달라도 완전히 별개의 객체(Object)로 취급하기 때문.
- 미등록 단어를 추론 Inference할 추가 로직이 전무.
- 학습 데이터에 존재하지 않던 새로운 표현이 입력되면 모델은 무조건 '모르는 단어'로 처리하며, 이는 일반화 성능의 급격한 하락으로 이어짐.

성능 평가 지표

```
정답: 온션뷰 가성비 숙소예요! 벌써 두 번째 방문이에요! 힐링하러 왔는데 완전 힐링하고 갑니다ㅎㅎ 뷰도 너무 예쁘고 시설도 깔끔하고 직원분들도 친절하세요
예측: 온션뷰 가성비 숙소예요! 벌써 두 편침 뽕물냄새앗! 힐링하러 왔는데 완전 힐링하고 갑니다ㅎㅎ 뷰도 너무 옛푸꿍 시설도 깔끔하고 직원분들도 친절하세요
```

Accuracy: 0.0000

Precision: 0.5156 | Recall: 0.5156 | F1: 0.5156

정확도 (Accuracy) 0.0000

- 문장 단위 완전 일치 → acc : 1.0
- 불일치 → acc : 0
- '온션뷰', '직건뽕물또' 등 사전에 없는 미등록 단어를 변환하지 못해 예측 문장과 정답 문장이 완전히 같지 않으므로 0점으로 처리됨.

F1 스코어 (F1 Score) 0.5156

- '가성비', '힐링하러 왔는데 완전', '뷰도 너무', '시설도 깔끔하고' 등 난독화되지 않고 원문과 형태가 동일하게 유지된 문자들이 다수 존재함.
- 전체 글자 중 약 51.5%가 교집합으로 인정되어 부분 점수를 획득함.

단어 사전(Dictionary) 기반으로 완전한 문장 복원에는 실패하여 acc는 0이지만, 문장에 섞여 있는 정상적인 단어들로 인해 F1스코어는 0.51로 도출

3-2. Dictionary Baseline 개선

딕셔너리 단어 정제

- 난독화된 단어와 딕셔너리에 존재하는 단어가 일치 할때 replace하는 것으로 복원을 진행 했음

문제

- 좋아요!, 좋아요~, 좋아요ㅎㅎ 들은 전부 같은 '좋아요'를 가리킴에도 공백 문자 단위로 tokenize 되다보니 다른 단어로 취급
 - 한글, 영문, 숫자, 공백 만 남기고, 단독 자음/모음과 기타 기호들 제거 필요'

- 원본 사전도 그대로 쓰되, 찾을 수 없을 경우 **Cleaned** 된 사전에서 한번 더 검색
- 쫘아요, 조하요, 줄알요, 죠하욧 또한 다르게 난독화된 '줄아요'이나 딕셔너리에 '쫘아요' 일부만 등록되어 있을 시 나머지는 '줄아요'로 복원되지 못함
 - 사전에 없는 단어가 들어오더라도 사전 내부에 있는 단어들 중 형태가 비슷한 단어를 찾아 치환 해주는 방식으로 개선 가능

해결

- 파이썬 정규 표현식 **re**라이브러리를 이용
 - <https://docs.python.org/ko/3/library/re.html>
- 노이즈 제거 함수 **remove_noise()**
 - 한글(완성형), 영대소문자, 숫자, 공백을 제외한 모든 문자(특수기호, ㅋㅋ, ㅎㅎ 등) 제거

```
cleaned_word = re.sub(r'^가-힣a-zA-Z0-9\s', '', word)
```

Fuzzy matching을 이용한 방식

사전에 없는 단어가 들어왔을 때 포기하지 않고, 사전 내부에 있는 단어들 중 형태가 가장 비슷한 단어를 찾아 **Mapping**

문제

- 딕셔너리를 개선하더라도 **exact matching**을 이용하므로 똑같은 의미의 단어(쫘아요, 조하요, 줄알요 등)더라도 딕셔너리에 없는 단어는 변환하지 못함.

해결

- 파이썬 내장 라이브러리인 **difflib**의 유사도 검사(레벤슈타인 거리 기반) 알고리즘을 활용
- 임계치(**Cutoff**) 이상의 유사도를 가진 단어로 강제 치환
- <https://rapidfuzz.github.io/RapidFuzz/> 이용
- **fuzzy** 알고리즘을 적용하기 위해 **difflib**의 **get_close_matches()**를 이용 했으나 **dictionary**의 모든 키와 입력 단어(**query**)를 일일이 파이썬 루프 내에서 비교하므로 시간이 매우 오래걸림. (30분 이상 연산 지속)
 - **c++** 기반 연산의 **rapidfuzz** 이용
 - <https://rapidfuzz.github.io/RapidFuzz/Usage/process.html#extractone>

```
choices (Collection[Sequence[Hashable]] | Mapping[Sequence[Hashable]]) -
list of all strings the query should be compared with or dict with a
mapping { : }
```

- `extractOne()`를 이용 , `process.extractOne` 내부의 `choices` 파라미터에 `CLEAN_KEYS`를 넘겨줌으로써 빠르게 `key` 검색
- 약 9분만에 결과 도출

3-2. 성능 요약

name	특징	Validation Accuracy	Validation CharF1
baseline	기본 dictionary 기반 replace	0.004882379050155349	0.4421080374996327
noise	특수 기호등의 노이즈 제거	0.004882379050155349	0.4500180711753619
fuzzy	fuzzy matching 이용해 문자열 유사도 검사	0.007989347536617843	0.5917188520400655

전체 동향

- 3가지 방식 모두 **Validation Accuracy**가 1% 미만(0.004~0.007)으로, 완벽한 문자열 일치(Exact match)를 달성하는 데 실패하고 있음.
- **CharF1**은 0.44~0.59 수준으로, 완벽히 맞추진 못해도 정답과 상당 부분 유사한 문자 구성을 예측해 내고 있음을 의미.

모델별 분석

Noise (특수 기호 및 단독 모음/자음 노이즈 제거)

- **Baseline** 대비 **Accuracy**는 완벽히 동일하며, **CharF1**만 소폭 상승(+0.0079).
- 단순 전처리만으로는 **Accuracy** 개선이 불가능하며, 이는 완벽하게 문장이 일치하도록 하지는 못함을 의미

Fuzzy (문자열 유사도 검사)

- **Baseline** 대비 **Accuracy** 약 1.6배 상승(+0.0031), **CharF1** 대폭 상승(+0.1496).
- 오타, 띄어쓰기, 변형 등을 유연하게 처리하는 **Fuzzy** 알고리즘 으로 '유연성 부족' 문제를 일부 해소 함으로써 좋은 성능 개선을 이끌어냄.

4. LLM Baseline 설명

<https://colab.research.google.com/drive/1UbTzH4sV8HGHXV6ZW-uyiyY35MiCfhHf#scrollTo=MJI5xdfrDDU1>

[선택한 모델 — `beomi/gemma-ko-2b`]

- 저희가 선택한 모델은 `beomi/gemma-ko-2b`입니다. Google의 Gemma 2B 모델을 한국어 데이터로 추가 학습시킨 한국어 특화 언어 모델

- 입니다. 파라미터 수는 약 20억 개로, Colab 무료 환경의 T4 GPU에서도 4bit 양자화를 적용하면 구동 가능한 적절한 크기입니다. 한국어 난독화 복원이라는 태스크 특성상 한국어 이해 능력이 중요하기 때문에, 범용 다국어 모델보다 한국어에 최적화된 이 모델을 선택했습니다.

[Tokenizer 구조]

- 토큰라이저는 텍스트를 모델이 이해할 수 있는 숫자 토큰으로 변환하는 역할을 합니다. Gemma는 **SentencePiece** 기반의 **BPE(Byte Pair Encoding)** 토큰라이저를 사용합니다. BPE는 자주 등장하는 문자 조합을 하나의 토큰으로 묶는 방식으로, 한국어처럼 형태소가 복잡한 언어도 효율적으로 처리할 수 있습니다.
- 코드에서는 `AutoTokenizer.from_pretrained`로 토큰라이저를 불러오고, `pad_token`을 `eos_token`으로 설정해 배치 처리 시 패딩 문제를 해결했습니다. 또한 `padding_side='right'`로 지정해 causal LM의 학습 방향과 일치시켰습니다.

[Encoder-Decoder 구조 vs Decoder-Only 구조]

- 언어 모델은 크게 Encoder-Decoder 구조와 Decoder-Only 구조로 나뉩니다.
- Encoder-Decoder는 T5, BART처럼 입력을 이해하는 인코더와 출력을 생성하는 디코더가 분리된 구조입니다. 번역이나 요약처럼 입력과 출력이 명확히 분리된 태스크에 적합합니다.
- 반면 Gemma는 **Decoder-Only** 구조입니다. 입력 프롬프트를 그대로 받아, 다음에 올 토큰을 순차적으로 예측하는 방식으로 텍스트를 생성합니다. GPT 계열이 이 구조에 해당하며, 최근 대부분의 LLM이 이 방식을 채택하고 있습니다. 저희 태스크처럼 프롬프트를 주고 이어서 복원된 문장을 생성하는 방식에는 Decoder-Only 구조가 자연스럽게 맞아떨어집니다.

[Pretrained의 의미]

- Pretrained, 즉 사전학습이란 모델이 방대한 텍스트 데이터를 통해 언어의 문법, 문맥, 의미 관계를 미리 학습해둔 상태를 말합니다. `beomi/gemma-ko-2b`는 이미 수십억 토큰 규모의 한국어 텍스트로 사전학습이 완료된 모델입니다.
- 덕분에 저희는 언어 이해 능력을 처음부터 학습시킬 필요 없이, 복원 태스크에 맞게 조금만 추가 학습하면 되는 상황입니다. 이것이 사전학습 모델을 활용하는 가장 큰 이점입니다.

[Fine-tuning 방식 — LoRA]

- 파인튜닝은 사전학습된 모델을 특정 태스크에 맞게 추가 학습하는 과정입니다. 다만 20억 파라미터 전체를 학습하는 건 Colab 환경에서 불가능하기 때문에, 저희는 **LoRA(Low-Rank Adaptation)** 기법을 사용했습니다.

- LoRA는 기존 모델의 가중치는 얼려두고, 각 레이어에 작은 저차원 행렬 두 개만 추가해서 학습하는 방법입니다. 전체 파라미터의 1% 미만만 학습해도 풀 파인튜닝에 근접한 성능을 낼 수 있습니다.
- 저희는 Attention 레이어인 `q_proj`, `k_proj`, `v_proj`, `o_proj`와 FFN 레이어인 `gate_proj`, `up_proj`, `down_proj`까지 총 7개 모듈을 타겟으로 지정했습니다. `rank`는 8, `alpha`는 32로 설정했으며, 추가로 메모리 절약을 위해 `paged_adamw_8bit` 옵티마이저를 사용했습니다.

[Generation 과정]

- 추론 시에는 HuggingFace의 `pipeline`을 사용해 텍스트를 생성합니다. 과정을 단계별로 설명하면 이렇습니다.
- 먼저 입력 난독화 문장을 `create_prompt` 함수로 프롬프트 형식에 맞게 가공합니다. 이 프롬프트를 토큰라이저가 토큰 ID 시퀀스로 변환하고, 모델은 이를 받아 다음 토큰을 하나씩 예측합니다. 이 과정을 반복해 완성된 출력 시퀀스가 만들어지면, 토큰라이저가 다시 텍스트로 디코딩합니다.
- 생성 옵션으로는 `do_sample=False`로 Greedy Decoding 방식을 선택했고, `max_new_tokens=50`으로 최대 출력 길이를 제한했습니다. 프롬프트 부분은 출력에서 제거하기 위해 `generated_text[len(prompt):]` 슬라이싱을 적용했습니다. 배치 추론 시에는 `batch_size=4`로 설정해 메모리와 속도 사이의 균형을 맞췄습니다.
-

5. 실험 결과 비교

아래 표를 반드시 포함한다.

실험 번호	모델	pretrained	Aug	Decodin g	LR	Batch Size	F1
Exp1							
Exp2							
Exp3							

6. 결과 해석

다음 질문에 답한다.

- Dictionary 방식과 LLM 방식의 차이는 무엇인가?
- 어떤 모델의 성능이 가장 높았는가?
- `augmentation`은 어떤 영향을 주었는가?
- `pretrained` 모델은 왜 성능이 높은가?

- 어떤 난독화 패턴이 가장 어려웠는가?
- 추가 성능 개선 방법은 무엇인가?

7. 팀원별 기여도

이름	담당 역할	실제 수행 내용	이해한 핵심 개념
김연수	팀원 A	데이터 분석 및 전처리	- 난독화된 한글 텍스트의 훼손 패턴을 탐색 및 분석하며 기존 단어 단위 매칭이 아닌 문자 단위(Character-level) 평가 방식이 자연어 처리에서 필수적 이유 이해 - 규칙 기반의 노이즈(특수문자 삽입, 띄어쓰기 변형)를 추가하는 데이터 증강(Augmentation) 파이프라인 구현, AI 모델의 일반화 성능을 높이기 위한 전처리 기법을 습득 - 원시 CSV 데이터를 Pandas로 다루며 HuggingFace Dataset 객체로 변환하고 학습/검증용으로 분할하는 과정을 통해, 실제 LLM 학습에 필요한 데이터 파이프라인의 전체적인 흐름을 체득함
이지현	팀원 B	- 규칙 기반 단어 사전을 구축하여 난독화 텍스트 복원 베이스라인(Baseline) 모델 설계 및 구현 - 정규 표현식 기반 노이즈 제거 로직 및 rapidfuzz를 활용한 퍼지 매칭 알고리즘 도입 - 검증 데이터를 통해 정확도와 문자 단위 F1 스코어산출 및 매칭 방식별 성능 개선 폭 정량 분석	- 완전 일치(Exact matching) 방식이 미등록 단어 및 신규 난독화 패턴에 취약하다는 규칙 기반 모델의 근본적 한계 파악 - 문맥을 고려하지 못하는 단어 독립적 치환 방식의 단점을 보완하기 위해, 문자열 유사도알고리즘이 복원 성능에 미치는 영향 이해 - 엄격한 척도인 정확도와 부분 일치를 반영하는 문자 단위 F1 스코어의 차이를 비교하며, 과제(Task) 목적에 부합하는 평가 지표(Evaluation metric) 선정의 중요성 체득

--	--	--	--

8. 생성형 AI 사용 기록

팀원 A 김연수

1. 어떤 질문을 했는가?

"Pandas DataFrame 형식으로 된 데이터를 HuggingFace의 Dataset 객체로 변환하고, 모델 검증을 위해 train 데이터와 validation 데이터를 8:2 비율로 분할하는 코드를 작성해 줘."

2. 어떤 코드 생성 또는 수정에 도움을 받았는가?

datasets 라이브러리의 Dataset.from_pandas() 메서드와 train_test_split() 함수를 사용하여 모델의 입력 형식에 맞게 데이터를 포매팅하고 분할하는 전체 전처리 코드 구조를 제공받았음.

3. AI가 생성한 코드 중 직접 수정한 부분은 무엇인가?

AI는 모든 데이터를 동일한 방식으로 변환하려 했으나, 평가용 데이터인 test.csv에는 정답 열(output)이 존재하지 않아 변환 시 KeyError가 발생함. 이를 해결하기 위해 테스트 데이터 변환 코드에서는 test_df[['input']]만 명시적으로 추출하도록 코드를 직접 수정함.

4. 이해되지 않았던 코드와 해결 과정은 무엇인가?

train_test_split()으로 데이터를 분할한 후 반환된 20%의 검증용 데이터가 split_dataset['test']라는 키 값으로 할당되는 것이 혼란스러웠음.

AI를 통해 HuggingFace 라이브러리의 내부 네이밍 규칙상 분할된 세트가 기본적으로 test로 명명된다는 점을 이해하고, 이를 val_data라는 직관적인 변수명으로 매핑하여 혼선을 방지하고자 하였음.

이지현

llm과는 달리 dictionary의 경우 개선을 어떻게 진행해야할지 아이디어를 얻기 위해 질문했습니다.

fuzzy 알고리즘을 적용하기 위해 difflib의 get_close_matches()를 이용했으나 dictionary의 모든 키와 입력 단어(query)를 일일이 파이썬 루프 내에서 비교하므로 시간이 매우 오래걸리는 문제를 해결하기 위해 관련 질문을 했습니다.

신연주

어떤 질문을 했는가???

토큰나이저 부분